

Тестирование и отладка многопоточных приложений

В этой главе

- Ошибки, связанные с конкурентностью.
- Поиск ошибок при тестировании и анализе кода.
- Разработка многопоточных тестов.
- Тестирование производительности многопоточного кода.

До сих пор основное внимание мы уделяли всему, что относится к созданию конкурентного кода, — доступному инструментарию, порядку его использования и общим вопросам конструкции и структур кода. Но одну из жизненно важных составляющих разработки такого кода пока еще не рассматривали — речь идет о тестировании и отладке. Если, читая эту главу, вы надеетесь узнать простой способ тестирования конкурентного кода, вы будете разочарованы. Тестирование и отладка конкурентного кода даются нелегко. Я же хочу познакомить вас с приемами, облегчающими решение этой задачи, а также рассмотреть вопросы, о которых стоит призадуматься.

Тестирование и отладка сродни двум сторонам медали — код подвергается тестированию для поиска возможных ошибок, а его отладка ведется с целью устранения этих ошибок. Если повезет, то вам пужно будет только устранить ошибки, найденные вашими собственными тестами, а не обнаруженные конечными пользователями вашего приложения. Прежде чем приступить к вопросам тестирования либо отладки, важно разобраться с возможными проблемами. Этим мы сейчас и займемся.

11.1. Типы ошибок, связанных с конкурентностью

При написании конкурентного кода вы можете столкнуться с любыми разновидностями ошибок, и это неудивительно. Но некоторые типы ошибок напрямую связаны с использованием конкурентности и имеют самое непосредственное отношение к теме данной книги. Обычно ошибки, связанные с конкурентностью, разбиваются на две категории, такие как:

- ❑ нежелательная блокировка;
- ❑ состояние гонки.

Это обобщенные категории, поэтому давайте разобьем их на части. Начнем с нежелательной блокировки.

11.1.1. Нежелательная блокировка

Что подразумевается под нежелательной блокировкой? Поток *блокируется* при отсутствии возможности продолжать выполнение, поскольку приходится чего-то ждать. Обычно это связано с чем-то вроде мьютекса, условной переменной или фьючерса, но причиной может быть и ожидание результатов операции ввода-вывода. Все это вполне естественные составляющие многопоточного кода, которые не всегда признаются желательными, отсюда и формулируется проблема нежелательной блокировки. В связи с этим возникает следующий вопрос: почему такая блокировка становится нежелательной? Как правило, это происходит из-за того, что некий другой поток также ждет освобождения заблокированного потока для выполнения какого-либо действия и оказывается также заблокированным. Существует несколько вариаций на эту тему.

- ❑ *Взаимная блокировка* — как следует из материала главы 3, в случае взаимной блокировки один поток чего-то ждет от другого потока, который, в свою очередь, чего-то ждет от первого потока. В наиболее наглядных проявлениях один из потоков, участвующих в этом, отвечает за пользовательский интерфейс, который в таком случае перестает отзываться. В иных случаях интерфейс сохранит отзывчивость, но ряд необходимых задач останется незавершенным, например, не произойдет возвращения из поиска или документ не пойдет на печать.
- ❑ *Активная блокировка* — явление, похожее на взаимную блокировку тем, что один поток ждет другого, который, в свою очередь, ждет его самого. Основное отличие здесь в том, что ожидание связано не с блокировкой, а с активным проверочным циклом, например со спин-блокировкой. В тяжелых случаях симптомы такие же, как и у взаимной блокировки (приложение зависает), за исключением высокого потребления ресурсов центрального процессора из-за непрекращающегося выполнения потоков, блокирующих друг друга. В менее тяжелых случаях активная блокировка когда-либо завершается по причине произвольного порядка диспетчеризации, но при выполнении задачи, попавшей в ситуацию активной блокировки, произойдет существенная задержка, в ходе которой будет высокий уровень потребления ресурсов центрального процессора.

- *Блокирование на операции ввода-вывода или на ожидании поступления внешних данных* — если поток заблокирован на ожидании поступления внешних данных, он не может продолжить выполнение, даже если данные вообще никогда не поступят. Поэтому нежелательно блокировать внешние данные потока, также выполняющего задачу, завершения которой могут дожидаться другие потоки.

Вот так вкратце можно охарактеризовать нежелательную блокировку. А как насчет состояния гонки?

11.1.2. Состояния гонки

Состояния гонки относятся к наиболее распространенным проблемам, связанным с многопоточным кодом, — многие взаимные и активные блокировки проявляются только из-за состояний гонки. Но проблемными являются не все состояния гонки. Состояние гонки возникает всякий раз, когда поведение зависит от относительной диспетчеризации операций в отдельно взятых потоках. Есть множество абсолютно безвредных состояний гонки. Например, по большому счету совершенно безразлично, какой именно рабочий поток станет работать с очередной задачей из очереди задач. Но с состояниями гонки связаны многие ошибки конкурентности. В частности, состояниями гонки зачастую вызываются следующие разновидности проблем.

- *Гонка за данными*. Это особый тип состояния гонки, приводящий к неопределенному поведению из-за несинхронизированного конкурентного доступа к совместно используемым областям памяти. Гонка за данными была представлена в главе 5 при рассмотрении модели памяти C++. Гонка за данными обычно возникает из-за некорректного использования атомарных операций для синхронизации потоков или из-за обращения к совместно используемым данным без блокировки соответствующего мьютекса.
- *Нарушение инвариантов*. Может проявляться в виде всяких указателей (из-за того, что другой поток удалил данные, к которым мы пытаемся обратиться), случайного повреждения памяти (из-за того, что поток читает данные, оказавшиеся несогласованными в результате частичного обновления) и двойного освобождения (например, когда два потока извлекают из очереди одно и то же значение и затем оба удаляют связанные с ним данные). Нарушения инвариантов могут быть связаны как с несоблюдением временных соотношений, так и с неверными значениями. Если операции в отдельно взятых потоках должны выполняться в определенном порядке, неправильная синхронизация может привести к состоянию гонки, из-за которой требуемый порядок иногда нарушается.
- *Проблемы времени жизни*. Несмотря на то что такие проблемы можно увязать в одно целое с нарушением инвариантов, они все же составляют отдельную категорию. Основная проблема, вызываемая ошибками в данной категории, заключается в том, что поток живет дольше тех данных, к которым обращается, и поэтому обращается к удаленным или же каким-либо образом разрушенным данным и, возможно, к хранилищу, которое даже повторно не используется для другого объекта. Обычно проблемы времени жизни возникают, когда поток ссылается на локальные переменные, которые вышли из области видимости еще до завершения

выполнения функции потока, но только этим сценарием они не ограничиваются. Если время жизни потока и время жизни данных, с которыми он работает, каким-то образом не связаны друг с другом, появляется возможность разрушения данных еще до завершения работы потока. Тем самым выбивается почва из-под ног у функции потока. Если самостоятельно вызвать функцию `join()`, чтобы дождаться завершения работы потока, нужно обеспечить невозможность обхода вызова `join()` при выдаче исключения. Применительно к потокам это составляет основу безопасности выдачи исключений.

Проблемные состояния гонки могут иметь самые печальные последствия. При возникновении взаимных и активных блокировок возможно следующее: приложение подвисает и ни на что не реагирует или же тратит слишком много времени на завершение задачи. Зачастую к запущенным процессам можно прикрепить отладчик, чтобы определить потоки, участвующие во взаимной или активной блокировке, и те объекты синхронизации, за которые ведется битва. В случаях гонки за данными, нарушений инвариантов и возникновения проблем времени жизни видимые симптомы (например, внезапные сбои или некорректные выходные данные) могут обнаруживаться в любом месте кода — программа может перенести содержимое памяти, используемой другой частью системы и востребуемой намного позже текущего момента. Сбой в таком случае проявится в той области кода, которая совершенно не связана с местом, содержащим ошибку, и вполне возможно, что он произойдет в ходе выполнения программы спустя значительное время после ошибочно выполненного действия. Это весьма актуально для систем с совместно используемой памятью — сколько бы ни прикладывать усилий по ограничению доступности данных из того или иного потока и по применению действенных механизмов синхронизации, любой поток может перенести данные, используемые любым другим потоком в этом же приложении.

Очертив круг интересующих нас проблем, посмотрим, что можно сделать для обнаружения и исправления имеющихся в коде ошибок.

11.2. Приемы обнаружения ошибок, связанных с конкурентностью

В предыдущем разделе мы рассмотрели типы ошибок, с которыми можно столкнуться при применении конкурентности, и с их возможными проявлениями в вашем коде. Усвоив эту информацию, можно проанализировать свой код на предмет тех мест, где вероятны такие ошибки, и продумать действия по обнаружению этих мест в конкретном разделе кода.

Пожалуй, самый простой способ — *визуальный анализ кода*. Но при всей своей кажущейся простоте провести такой анализ в должной мере весьма непросто. Читая свой же ранее написанный код, проще всего разглядеть свои же намерения при его написании, а не то, что получилось на самом деле. Аналогично этому при изучении кода, написанного кем-то другим, мы стремимся быстро просмотреть его, проверяя на соответствие местным стандартам программирования. Отмечаем при этом наиболее очевидные проблемные места. Нужно не пожалеть времени на прочесывание

кода мелким гребнем, чтобы обнаружить проблемы, связанные с конкурентностью, а заодно и те, что не связаны с ними. (Все это можно совместить, поскольку устранению подлежат любые ошибки.) На что конкретно нужно будет обратить внимание, станет понятно из предстоящего вскоре анализа кода.

Но даже после тщательного исследования кода некоторые ошибки можно и проглядеть. В любом случае для своего же спокойствия не остается ничего иного, как подтвердить работоспособность этого кода. Следовательно, продолжая рассматривать вопросы визуального исследования кода, мы перейдем к изучению приемов, применяемых при тестировании многопоточных программ.

11.2.1. Просмотр кода с целью поиска возможных ошибок

Как уже упоминалось, поиск ошибок, связанных с конкурентностью, при визуальном просмотре многопоточного кода требует особой тщательности. По возможности нужно привлечь к этой работе и других специалистов. Поскольку они не имеют отношения к созданию этого кода, станут разбираться в том, как он работает, что поможет обнаружить любые ошибки в коде. Важно, чтобы у них было достаточно времени для качественного изучения кода, а не пара минут, чтобы взглянуть вскользь. Здесь нужен серьезный, вдумчивый анализ. Многие ошибки, связанные с конкурентностью, не бросаются в глаза при беглом просмотре, их выявление требует кропотливого изучения кода.

У привлекаемых к просмотру кода коллег будет свежий взгляд. Они будут на все смотреть с другой позиции и смогут заметить то, что ускользает от вашего взгляда. Если таких коллег у вас нет, попросите друзей или даже просто опубликуйте код в Интернете (постарайтесь при этом не расстраивать юристов вашей компании). Если привлечь к просмотру кода так никого и не удастся или привлеченные специалисты ничего не найдут, не стоит огорчаться, еще не все потеряно. Для начала было бы полезно на некоторое время оставить код в покое и поработать над другой частью приложения, почитать книгу или отправиться на прогулку. Если сделать перерыв, то подсознание может работать над решением проблем в фоновом режиме, пока будет сосредоточено на чем-то другом. Кроме того, код, когда вы к нему вернетесь, утратит узнаваемость и вы посмотрите на него свежим взглядом.

Можно и не привлекать для просмотра сторонних специалистов, обойдясь собственными силами. Один из эффективных приемов — попробовать подробно объяснить принципы его работы каким-нибудь благодарным слушателем. Они могут быть даже неодушевленные предметы, у многих команд для этого есть игрушечные медвежата или пылята. Лично я считаю чрезвычайно полезным написание подробных пояснений к коду. Объясняя предназначение кода, следует уделять внимание каждой строке, тому, что должно произойти, к каким данным происходит обращение и т. д. Задайте самим себе вопросы по коду и дайте на них развернутые ответы. Я считаю такое занятие весьма эффективным приемом — задавая самому себе вопросы и тщательно обдумывая ответы, зачастую удается выявить наличие той или иной проблемы. Польза от таких вопросов проявится не только при просмотре собственного кода, но и при изучении любого другого кода.

Вопросы, требующие осмысления при просмотре многопоточного кода

Как уже упоминалось, рецензенту (автору кода или стороннему специалисту) следует задуматься о конкретных вопросах применительно к просматриваемому коду. Эти вопросы помогут сконцентрировать внимание рецензента на соответствующих особенностях кода и помочь выявлению потенциальных проблем. Ниже приведу перечень вопросов, которые я привык задавать, хотя он, скорее всего, не исчерпывающий. Вы можете поднять другие вопросы, которые помогут лучше сконцентрироваться на поиске ошибок. А вот так выглядят мои вопросы.

- ☐ Какие данные нуждаются в защите от конкурентного доступа?
- ☐ Как можно обеспечить защищенность данных?
- ☐ Какие фрагменты кода будут выполняться в это же время в других потоках?
- ☐ Какие мьютексы будут удерживаться анализируемым потоком?
- ☐ Какие мьютексы могут удерживаться другими потоками?
- ☐ Существуют ли какие-нибудь требования к порядку выполнения операций в анализируемом потоке, а также в других потоках? Как эти требования выполняются?
- ☐ Являются ли данные, загружаемые анализируемым потоком, по-прежнему действительными? Могли ли они быть изменены другими потоками?
- ☐ Если предположить, что другой поток мог изменить данные, то какое бы это имело значение и как можно было бы гарантировать, что такое никогда не случится?

Последний вопрос — мой любимый, поскольку он заставляет меня задуматься о взаимоотношениях между потоками. Предполагая наличие ошибки, связанной с конкретной строкой кода, можно далее выступить в роли сыщика и отследить причину. Чтобы убедиться в отсутствии ошибок, нужно рассмотреть каждый крайний случай и возможный порядок выполнения операций. Это особенно актуально при защите данных за время их жизни более чем одним мьютексом, как в случае с потокобезопасной очередью из главы 6, где для головы и хвоста очереди имелись отдельные мьютексы: чтобы обеспечить безопасность обращения при удержании одного мьютекса, нужно быть уверенными, что поток, удерживающий *другой* мьютекс, не может обратиться к тому же элементу. Также вполне очевидно, что особому анализу должны подвергаться общедоступные данные или те данные, на которые другой код может легко получить указатель или ссылку.

Важен и предпоследний вопрос приведенного выше перечня, поскольку он касается весьма распространенной и часто допускаемой ошибки: если освободить, а затем снова получить мьютекс, следует предполагать, что совместно используемые данные могли претерпеть изменения со стороны других потоков. При всей очевидности такого предположения, если блокировки мьютексов не лежат на поверхности (возможно, из-за того, что они выполняются внутри объекта), неосознанно допускается именно такая ошибка. В главе 6 мы продемонстрировали, как излишняя детализация обращения к данным в функциях, предоставляемых потокобезопасной структуре данных, может приводить к состоянию гонки. Если для стека, не являющегося потокобезопасным, наличие отдельных операций `top()` и `pop()` имеет

вполне определенный смысл, то для стека, допускающего конкурентное обращение со стороны сразу нескольких потоков, такой вариант уже не подходит, поскольку блокировка на внутреннем мьютексе между двумя вызовами освобождается, позволяя вносить изменения со стороны другого потока. Как вы видели в главе 6, решение заключается в объединении двух операций, чтобы обе они выполнялись под защитой блокировки одного и того же мьютекса, не позволяя возникать состоянию гонки.

Итак, вы (или кто-то другой) просмотрели код. Вы уверены, что ошибок нет. Но, как говорится, распробовать пудинг можно только в еде, так же нужно и протестировать код, чтобы убедиться в отсутствии ошибок.

11.2.2. Обнаружение ошибок, связанных с конкурентностью, путем тестирования

При разработке однопоточных приложений процедура их тестирования при наличии достаточного времени относительно проста. В принципе, можно определить все возможные наборы входных данных (или по крайней мере те, что представляют интерес) и прогнать их через приложение. Если приложение демонстрирует правильное поведение и производит верные выходные данные, то возникает уверенность в его работоспособности с этим набором входной информации. Тестирование на выход из состояний ошибок, например на обработку ошибок переполнения диска, дается сложнее, но замысел прежний: установка исходных условий и предоставление приложению возможности справиться с ними.

Тестирование многопоточного кода на порядок сложнее, поскольку точный порядок диспетчеризации потоков определению не поддается и от запуска к запуску может быть разным. Следовательно, даже если запускать приложение с одними и теми же входными данными, то порой оно может работать правильно, а порой, при наличии скрытых в коде предпосылок к состояниям гонки, давать сбой. Наличие таких предпосылок означает, что сбой в коде будет появляться не всегда, а только время от времени.

Учитывая сложность воспроизводства ошибок, связанных с конкурентностью, тесты требуют тщательной проработки. Нужно, чтобы в каждом тесте запускался наименьший объем кода, в котором может проявиться ошибка. Так можно будет наилучшим образом изолировать сбойный код, если тест не будет пройден — лучше провести непосредственное тестирование одновременно используемой очереди, чтобы проверить, что одновременное помещение в нее данных и их одновременное извлечение работают, а не тестировать ее в контексте всего кода, где она применяется. При разработке кода полезно задуматься и о том, как он будет тестироваться, разработку с прицелом на тестируемость рассмотрим в этой главе чуть позже.

Чтобы проверить, связана ли проблема именно с конкурентностью, стоит убрать из теста все, что с ними связано. Если проблема проявляется при запуске всего кода в одном потоке, значит, причина кроется в самой банальной ошибке, а не в ошибке, связанной с конкурентностью. Это особенно важно при попытке отслеживания той ошибки, что проявляется при обычных прогонах программы, а не в хитросплетениях используемых тестов. Лишь то, что ошибка проявляется в многопоточной части вашего приложения, еще не означает, что она автоматически становится связанной

с конкурентностью. Если для управления уровнями конкурентности используются пулы потоков, то обычно для указания количества рабочих потоков можно воспользоваться параметром конфигурации. Если потоки управляются не в автоматическом режиме, то для использования в тесте одного потока придется вносить изменения в код. В любом случае, если сократить фронт выполнения приложения до одного потока, появится возможность устранить конкурентность как причину возникновения ошибки. С другой стороны, проблема исчезает в *одноядерной системе* (даже при запуске нескольких потоков), но появляется в *многоядерных* или *многопроцессорных системах*. В этом случае речь может идти о состоянии гонки или же проблемах синхронизации либо порядка доступа к памяти.

При тестировании конкурентного кода важна не только структура тестируемого кода, но и структура теста и среды тестирования. Если продолжить разбор примера тестирования конкурентной очереди, то следует продумать целый ряд различных сценариев.

- ☐ Один поток автономно вызывает `push()` или `pop()` с целью проверки работоспособности очереди на базовом уровне.
- ☐ Один поток вызывает `push()` в отношении пустой очереди, в то время как другой поток вызывает `pop()`.
- ☐ Несколько потоков вызывают `push()` в отношении пустой очереди.
- ☐ Несколько потоков вызывают `push()` в отношении заполненной очереди.
- ☐ Несколько потоков вызывают `pop()` в отношении пустой очереди.
- ☐ Несколько потоков вызывают `pop()` в отношении заполненной очереди.
- ☐ Несколько потоков вызывают `pop()` в отношении частично заполненной очереди с недостаточным количеством элементов для всех потоков.
- ☐ Несколько потоков вызывают `push()`, в то время как один поток вызывает `pop()` в отношении пустой очереди.
- ☐ Несколько потоков вызывают `push()`, в то время как один поток вызывает `pop()` в отношении заполненной очереди.
- ☐ Несколько потоков вызывают `push()`, в то время как несколько потоков вызывают `pop()` в отношении пустой очереди.
- ☐ Несколько потоков вызывают `push()`, в то время как несколько потоков вызывают `pop()` в отношении заполненной очереди.

Обдумав все эти сценарии (и не только их), нужно рассмотреть дополнительные факторы, касающиеся среды тестирования.

- ☐ Что подразумевается под «несколькими потоками» в каждом случае (это 3, 4, 1024 потока)?
- ☐ Достаточно ли вычислительных ядер в системе, чтобы запустить каждый поток в своем собственном ядре?
- ☐ На каких процессорных архитектурах должны запускаться тесты?
- ☐ Как обеспечить подходящий режим диспетчеризации для той части тестов, которая обозначена фразой «в то время как»?

Применительно к вашей ситуации следует обдумать и дополнительные факторы. Из четырех приведенных выше аспектов среды первый и последний относятся к структуре самого теста (и рассматриваются в подразделе 11.2.5), а другие два относятся к той физической системе, которая используется для тестирования. Количество используемых потоков зависит от конкретного тестируемого кода, но существуют различные способы структурирования теста для получения подходящего порядка диспетчеризации. Прежде чем приступить к рассмотрению этих способов, давайте посмотрим, как можно сконструировать код приложения, чтобы его легче было тестировать.

11.2.3. Разработка кода с прицелом на удобство тестирования

Тестирование многопоточного кода дается нелегко, поэтому возникает желание облегчить эту задачу. Самое важное из возможных действий в этом направлении — разработка кода с прицелом на удобство его тестирования. Существует множество публикаций применительно к разработке такого кода для однопоточной среды, и многие приводимые там советы не потеряли своей актуальности и в нашем случае. В общем, код проще поддается тестированию, если соблюдены следующие факторы:

- ❑ четкое определение сфер ответственности каждой функции и класса;
- ❑ краткость и конкретность определения функций;
- ❑ возможность установить со стороны тестов полный контроль над средой окружения тестируемого кода;
- ❑ сосредоточенность в одном месте, а не разбросанность по всей системе кода операции, подвергаемой тестированию;
- ❑ продумывание порядка тестирования кода до его написания.

Все эти факторы актуальны и для многопоточного кода. В сущности, можно сказать, что обращать внимание на пригодность к тестированию многопоточного кода еще важнее, чем однопоточного, поскольку его по определению гораздо сложнее тестировать. Также важен и последний фактор: даже если вы еще не дошли до того, чтобы создавать тесты до написания самого тестируемого кода, то, прежде чем приступить к написанию кода, стоит подумать о том, как можно будет его протестировать: какие входные данные использовать, при каких условиях, скорее всего, станут возникать проблемы, как можно спровоцировать код на проявление проблем и т. д.

Суть одного из лучших способов создания конкурентного кода с прицелом на тестирование заключается в исключении конкурентности. Если код можно разбить на части, которые отвечают за пути передачи данных между потоками, и на части, которые работают с передаваемыми данными в рамках одного потока, то задача существенно упростится. Те части приложения, что работают с данными, обращение к которым бывает только из одного потока, теперь можно протестировать с использованием обычных однопоточных приемов тестирования. Сложный для тестирования конкурентный код, занимающийся обменом данными между потоками

и обеспечивающий доступность к конкретному блоку данных в любой момент со стороны только одного потока, теперь стал намного меньше и легче поддающимся тестированию.

Например, если приложение разработано в качестве многопоточного конечного автомата, его можно разбить на несколько частей. Логике состояния для любого потока, обеспечивающего для каждого набора входных событий корректность переходов и операций, можно протестировать независимо и с применением однопоточных приемов, с помощью тестового набора, предоставляющего входные события, которые будут поступать от других потоков. Затем можно независимо протестировать основной код конечного автомата и код маршрутизации сообщений, обеспечивающий правильную доставку событий нужным потокам и в нужном порядке, но с несколькими конкурентными потоками и с простой логикой состояния, специально разработанной для тестов.

Или же, если есть возможность разбить код на несколько блоков — *чтения совместно используемых данных/преобразования данных/обновления совместно используемых данных*, — можно будет протестировать ту часть кода, которая занимается преобразованием данных, воспользовавшись всеми обычными однопоточными приемами, поскольку теперь это однопоточный код. Сложная задача тестирования многопоточных преобразований теперь сведется к тестированию чтения и обновлению совместно используемых данных, что представляется куда более простой задачей.

Обратите внимание, что в библиотечных вызовах для хранения состояния допускается применение внутренних переменных, которые затем станут использоваться совместно, если один и тот же набор библиотечных вызовов будет задействован сразу несколькими потоками. Это может стать проблемой, потому что факт обращения кода к совместно используемым данным вскроется не сразу. Но со временем станет понятно, какие это библиотечные вызовы, они будут сразу бросаться в глаза. Затем можно либо добавить соответствующую защиту и синхронизацию, либо воспользоваться альтернативной функцией, не представляющей опасности при конкурентном доступе из нескольких потоков.

Разработка легко поддающегося тестированию многопоточного кода не ограничивается структурированием кода с целью минимизации объема кода, необходимого для решения вопросов, связанных с конкурентностью, и контролем использования библиотечных вызовов, не обладающих потокобезопасностью. Полезно помнить о вопросах из подраздела 11.2.1, которые следует задавать самим себе при просмотре кода. Хотя эти вопросы и не относятся напрямую к тестированию и готовности кода к тестированию, но, если переосмыслить вопросы с прицелом на тестирование и подумать о способах тестирования кода, ответы на них окажут благоприятное влияние на решения в выборе конструкции, что облегчит тестирование.

Теперь, когда рассмотрены вопросы разработки кода с прицелом на облегчение тестирования и потенциально модифицированный код для отделения «конкурентных» частей (например, потокобезопасных контейнеров или логики событий конечного автомата) от «однопоточных» частей (которые по-прежнему могут взаимодействовать с другими потоками при содействии конкурентных частей кода), давайте посмотрим на приемы тестирования конкурентного кода.

11.2.4. Приемы тестирования многопоточного кода

Итак, сценарий тестирования продуман, создан небольшой фрагмент кода для испытания тестируемых функций. И как теперь добиться выполнения любых потенциально проблематичных последовательностей диспетчеризации с целью избавления от ошибок?

Способов здесь несколько, и для начала рассмотрим принудительное тестирование, или стресс-тестирование.

Стресс-тестирование

Принудительное тестирование предусматривает создание для кода стрессовой обстановки с целью его испытания на прочность. Обычно под этим подразумевается многократный запуск кода, возможно, с многочисленными одновременно запускаемыми потоками. Если есть ошибка, проявляющаяся только при определенном порядке диспетчеризации потоков, то чем больше запусков кода, тем выше вероятность проявления ошибки. Когда тест запускается один раз и заканчивается успешным прохождением, может появиться уверенность в работоспособности кода. Если его прогнать подряд десять раз и всякий раз успешно, уверенность возрастет. При успешном прогоне теста миллиард раз уверенность станет еще крепче.

Возникающая в результате тестирования уверенность не зависит от объема кода, прогоняемого через каждый тест. Если детализация тестирования достаточно высока, вроде тех тестов, которые упоминались ранее в отношении потокобезопасной очереди, такое стресс-тестирование может дать вам высокую степень уверенности в работоспособности кода. С другой стороны, если тестируемый код значительно больше по объему, количество возможных перестановок при диспетчеризации потоков настолько велико, что даже миллиардное число тестовых прогонов не поможет избавиться от низкого уровня уверенности в работоспособности кода.

Недостаток стресс-тестирования заключается в том, что оно может дать ложную уверенность в работоспособности кода. Если при создании теста исходить из того, что проблемные обстоятельства никогда не возникнут, можно выполнять прогон теста вплоть до бесконечности и он не будет давать сбой, даже если неизменно будет сбывать при слегка измененных обстоятельствах. Наихудший вариант такого развития событий возникает, когда система, на которой производится тестирование, настроена так, что проблематичные условия в принципе невозможны. Так бывает, когда производственная система отличается от тестовой, и конкретное сочетание оборудования и операционной системы не дает материализоваться условиям, при которых возникает ошибка.

Классический пример — тестирование многопоточного приложения на однопроцессорной системе. Поскольку каждый поток приходится запускать на одном и том же процессоре, весь код подвергается автоматической сериализации, и многие проблемы, связанные с состояниями гонки и перебрасыванием данных в кэш-памяти, с которыми можно столкнуться в многопроцессорной системе, просто исчезают. Но это еще не все, процессоры с разной архитектурой предоставляют различные средства диспетчеризации и упорядочения доступа к памяти. Например, на архитектурах x86 и x86-64 атомарные операции загрузки всегда одинаково-

вы, независимо от того, какая семантика применена — `memory_order_relaxed` или `memory_order_seq_cst` (см. подраздел 5.3.3). Это означает, что код, написанный с расслабленной семантикой упорядочения доступа к памяти, может работать на архитектуре x86, а вот в системах с более детализированным набором инструкций упорядочения доступа к памяти, например на SPARC, он даст сбой.

Если от приложения требуется переносимость в широком диапазоне целевых систем, важно будет протестировать его на показательных экземплярах таких систем. Именно поэтому в подразделе 11.2.2 в перечень учитываемых факторов включена архитектура процессора, используемого для тестирования.

Для успешного стресс-тестирования очень важно избавиться от ложной уверенности. Для этого следует тщательно продумать конструкцию теста не только в плане выбора тестируемого блока кода, но и в отношении конструкции средства тестирования и выбора среды тестирования. Нужно обеспечить тестирование максимально возможного количества путей выполнения кода и моментов взаимодействия потоков. И это еще не все. Следует знать, какие варианты уже охвачены тестированием, а какие — еще нет.

Стресс-тестирование придает некую степень уверенности в работоспособности кода, но оно не гарантирует выявления всех проблем. Если есть время и соответствующие программные средства, то для поиска проблем к коду можно применить еще один прием. Я называю его *комбинационным имитационным тестированием*.

Комбинационное имитационное тестирование

Название воспринимается с трудом, поэтому поясню, что имеется в виду. Идея в том, чтобы запускать тестируемый код со специальным фрагментом программы, *имитирующим* прогон в реальных условиях среды применения. Возможно, вам известны программные средства, позволяющие запускать на одном физическом компьютере сразу несколько виртуальных машин, где характеристики виртуальной машины и ее оборудования имитируются управляющей программой. Здесь замысел тот же, только вместо эмуляции системы программа имитации записывает последовательность обращения к данным, блокировок и атомарных операций из каждого потока. Затем она применяет правила модели памяти C++ для повторения прогона с каждой допустимой *комбинацией* операций и выявления состояний гонки и взаимных блокировок.

Хотя такое исчерпывающее комбинационное тестирование гарантирует обнаружение всех проблем, на которые нацелена конструкция системы, для любых, кроме самых простых, программ оно займет уйму времени, поскольку количество комбинаций с увеличением числа потоков и числа операций, выполняемых каждым потоком, растет в геометрической прогрессии. Этот прием лучше приберечь для более детализированного тестирования отдельных частей кода, а не для всего приложения. Еще один явный недостаток — зависимость этого приема от доступности программы имитации, способной управлять операциями, используемыми в вашем коде.

Итак, есть прием, предполагающий многократный прогон теста при обычных условиях, но способный проглядеть проблему, и есть прием, предполагающий многократный прогон теста при особых условиях, но с высокой вероятностью обнаружения всех имеющихся проблем. А существуют ли другие варианты?

Третий вариант предполагает использование библиотеки, обнаруживающей проблемы по мере их проявления при прогоне теста.

Обнаружение проблем, проявляющихся при прогоне теста с использованием специализированной библиотеки

Хотя этот вариант не предоставляет исчерпывающей проверки, присущей комбинационному имитационному тестированию, специализированная реализация библиотечных элементов синхронизации, включая мьютексы, блокировки и условные переменные, позволяет выявить многие проблемы. Например, зачастую требуется, чтобы все обращения к области совместно используемых данных осуществлялись с блокировкой конкретного мьютекса. Если можно проверить, какие мьютексы были заблокированы при обращении к данным, появится возможность проконтролировать факт блокировки соответствующего мьютекса путем вызова потока при обращении к данным и отправки сообщения о сбое, если этот факт не подтвердится. Помечая каким-то образом совместно используемые данные, можно позволить библиотеке провести для вас такую проверку.

Эта реализация библиотеки может также записывать последовательность блокировок, если конкретным потоком одновременно удерживается сразу несколько мьютексов. Если другой поток блокирует те же самые мьютексы в ином порядке, это может быть зарегистрировано как *потенциальная* взаимная блокировка, даже если тест при прогоне с ней не столкнулся.

Еще в одном типе специализированной библиотеки, применяемой при тестировании многопоточного кода, реализация элементов потока, таких как мьютексы и условные переменные, предоставляет создателю тестов возможность брать контроль над тем, какому потоку получать блокировку, когда в режиме ожидания находятся сразу несколько потоков или какой поток уведомляется путем вызова функции `notify_one()` в отношении условной переменной. Это позволит задавать конкретные сценарии и проверять, оправдывает ли работа кода по таким сценариям возложенные на него ожидания.

Некоторые из рассмотренных средств тестирования следовало бы поставлять как часть стандартной библиотеки C++, а остальные можно создать в качестве надстройки над стандартной библиотекой в виде части вашей конструкции средства тестирования.

Рассмотрев различные приемы выполнения тестового кода, перейдем к способам структурирования кода для получения желаемого порядка диспетчеризации потоков.

11.2.5. Структурирование многопоточного тестового кода

В подразделе 11.2.2 говорилось о необходимости поиска способов предоставить подходящую диспетчеризацию потоков для той части тестов, которая шла за фразой «в то время как». Настало время рассмотреть вопросы, связанные с решением этой задачи.

Прежде всего нужно решить вопрос с организацией набора потоков, каждый из которых будет выполнять выбранный фрагмент кода в указанный момент времени. В общем случае речь идет о двух потоках, но все можно легко расширить

и на большее число потоков. На первом этапе следует разбить каждый тест на отдельные части:

- ❑ код общей настройки, который должен выполняться в самом начале;
- ❑ потоковый код настройки, который должен исполняться в каждом потоке;
- ❑ код, запускаемый в конкурентных потоках;
- ❑ код, предназначенный для запуска после завершения конкурентного выполнения, который может включать утверждения о состоянии кода.

В целях пояснения рассмотрим конкретный пример из перечня сценариев тестирования, приведенного в подразделе 11.2.2: один поток вызывает функцию `push()` в отношении пустой очереди, в то время как другой поток вызывает функцию `pop()`.

Общий код настройки прост: нужно создать очередь. У потока, выполняющего функцию `pop()` кода, настройки применительно к потоку нет. Код настройки применительно к потоку, выполняющему функцию `push()`, зависит от интерфейса очереди и типа сохраняемого в ней объекта. Если сохраняемый объект трудоемок в создании или требует размещения в куче, то всю предварительную подготовку желательно провести в виде части настройки применительно к потоку, чтобы не оказывать негативного влияния на тест. С другой стороны, если в очереди просто сохраняются обычные `int`-объекты, то от создания `int`-значений в коде настройки мы ничего не выиграем. Тестируемый код относительно прост — вызов `push()` из одного потока и вызов `pop()` из другого потока, ну а как насчет кода «после завершения»?

В данном случае все зависит от того, что требуется получить от функции `pop()`. Если предполагается блокировка до появления в очереди данных, то абсолютно ясно: хочется увидеть, что возвращаются те самые данные, которые были предоставлены вызову функции `push()`, после чего очередь становится пустой. Если функция `pop()` не блокируется и может завершиться, даже если очередь пуста, нужно протестировать две возможности: либо `pop()` возвращает элемент данных, предоставленный функции `push()`, и очередь становится пустой, либо `pop()` сигнализирует об отсутствии данных в очереди, а в очереди имеется один элемент. Должно произойти либо одно, либо другое; хотелось бы избежать сценария, при котором `pop()` сигнализирует, что «нет данных», а очередь пуста, либо сценария, при котором `pop()` возвращает значение, а очередь все же не пуста. Чтобы упростить тест, предположим, что мы имеем дело с блокирующей функцией `pop()`. В таком случае в завершающем коде должно быть утверждение, что извлеченное значение является тем же самым, что и помещенное значение, и очередь пуста.

После определения различных частей кода нужно приложить все усилия для обеспечения их запуска по плану. Для этого можно воспользоваться набором промисов `std::promise`, показывающих момент готовности всех частей. Каждый поток выдает промис, чтобы показать, что он готов, а затем ждет на фьючерсе `std::shared_future` (вернее, на его копии), полученном из третьего промиса `std::promise`, а основной поток ждет выдачи промисов от всех потоков и затем запускает потоки на выполнение с помощью сигнала `go`. Тем самым гарантируется, что каждый поток запущен и находится в точке, непосредственно предшествующей коду, который должен выполняться параллельно; весь потоковый код настройки должен завершиться до установки промиса `go`. Наконец, главный поток ждет завершения других потоков

и проверяет получившееся состояние. И наконец, основной поток ждет, пока потоки завершат свою работу, и проверяет конечное состояние. Следует также не забывать об исключениях и убедиться в отсутствии потоков, ожидающих сигнала `go`, выдачи которого не будет. Один из путей структурирования этого теста показан в листинге 11.1.

Листинг 11.1. Пример теста для конкурентных вызовов `push()` и `pop()` в отношении очереди

```
void test_concurrent_push_and_pop_on_empty_queue()
{
    threadsafe_queue<int> q;
    std::promise<void> go, push_ready, pop_ready;
    std::shared_future<void> ready(go.get_future());
    std::future<void> push_done;
    std::future<int> pop_done;
    try
    {
        push_done=std::async(std::launch::async,
                             [&q,ready,&push_ready]()
                             {
                                 push_ready.set_value();
                                 ready.wait();
                                 q.push(42);
                             });
        pop_done=std::async(std::launch::async,
                             [&q,ready,&pop_ready]()
                             {
                                 pop_ready.set_value();
                                 ready.wait();
                                 return q.pop();
                             });
        push_ready.get_future().wait();
        pop_ready.get_future().wait();
        go.set_value();
        push_done.get();
        assert(pop_done.get()==42);
        assert(q.empty());
    }
    catch(...)
    {
        go.set_value();
        throw;
    }
}
```

Структура в значительной степени совпадает с ранее рассмотренной. Сначала как часть общей настройки создается пустая очередь ❶. Затем создаются все промисы для сигналов «готовности» ❷ и берется фьючерс `std::shared_future` для сигнала `go` ❸. После этого создается фьючерс, который будет использоваться для обозначения завершения работы потоков ❹. Это нужно сделать вне блока `try`, чтобы можно было установить сигнал `go` на исключение без ожидания завершения работы тестовых потоков (что привело бы к взаимной блокировке, которая в коде теста весьма нежелательна).

Затем внутри блока `try` можно запустить потоки (❸ и ❹) — использование `std::launch::async` гарантирует, что каждая задача выполняется в своем собственном потоке. Заметьте, что применение `std::async` облегчает задачу обеспечения безопасности выдачи исключений по сравнению с использованием простого объекта `std::thread`, поскольку деструктор для фьючерса присоединится к потоку. Захваты в лямбда-функциях указывают на то, что каждая задача для подачи сигнала о готовности будет ссылаться на очередь и соответствующий промис, забирая копию фьючерса `ready`, полученную из промиса `go`.

В соответствии с предыдущим описанием каждой задачей устанавливается свой собственный сигнал `ready`, после чего перед запуском тестового кода ожидается поступление общего сигнала `ready`. Основной поток делает все наоборот — ждет сигналов от обоих потоков ❸, прежде чем подать им сигнал на запуск настоящего теста ❹.

И наконец, основной поток вызывает функцию `get()` в отношении фьючерсов из асинхронных вызовов, чтобы дождаться завершения задач, ❿ и ⓫, и проверяет результаты. Следует отметить, что операция `pop` извлекает возвращаемое значение из фьючерса ❼, чем можно воспользоваться для получения результата для утверждения ⓫.

Если выдается исключение, устанавливается сигнал `go`, чтобы исключить любую возможность получения висячего потока, и происходит повторная выдача исключения ⓫. Фьючерсы, соответствующие задачам ❹, были объявлены последними, поэтому они будут уничтожены первыми, а их деструкторы будут ждать завершения задач, если этого еще не произошло.

Несмотря на создающееся впечатление, что для тестирования двух простых вызовов здесь слишком много шаблонного кода, чтобы рассчитывать на успешное тестирование пугного вам кода, необходимо воспользоваться чем-то подобным. Например, запуск потока может занять слишком много времени, поэтому если не заставить потоки ждать сигнала `go`, то поток, помещающий данные, может завершиться еще до запуска потока, извлекающего данные, что полностью лишит тест всякого смысла. Данный способ использования фьючерсов гарантирует, что оба потока запускаются и блокируются на одном и том же фьючерсе. Затем разблокировка фьючерса позволит потокам приступить к выполнению. После того как эта структура будет освоена, задача создания новых тестов по той же схеме не составит особого труда. Для тестов, требующих использования более двух потоков, эта схема допускает расширение и на дополнительные потоки.

До сих пор мы говорили лишь о *корректности* многопоточного кода. Это, конечно, наиболее важный, но не единственный вопрос, ради которого проводится тестирование. Важно также протестировать *производительность* многопоточного кода, о чем пойдет речь далее.

11.2.6. Тестирование производительности многопоточного кода

Одной из основных причин, по которым для приложения делается выбор в пользу применения конкурентности, является возможность повышения производительности приложений в условиях роста распространенности многоядерных процессоров. Поэтому важно протестировать создаваемый вами код на подтверждение факта роста

его производительности, как это делалось при любых других попытках проведения оптимизации кода.

Особый интерес при использовании конкурентности для повышения производительности представляет вопрос *масштабируемости* — вам нужен код, который при прочих равных условиях на 24-ядерной машине работает примерно в 24 раза быстрее или обрабатывает в 24 раза больше данных, чем на одноядерной машине. Вам не нужен код, работающий вдвое быстрее на двухъядерной машине, но медленнее на 24-ядерной машине. В подразделе 8.4.2 было показано, что если значительная часть кода запускается только в одном потоке, то возможный прирост производительности будет ограничен. Поэтому перед началом тестирования стоит изучить общую конструкцию кода, чтобы понять, стоит ли надеяться на 24-кратное увеличение производительности, или последовательно выполняемая часть этого кода предопределяет ограничение увеличения максимум до трехкратного.

В предыдущих главах уже говорилось, что конкуренция процессоров за доступ к структуре данных может серьезно повлиять на производительность. То, что прекрасно масштабируется на малое количество процессоров, может утратить производительность при существенном росте их числа из-за резкого повышения уровня конкуренции.

Соответственно, при тестировании многопоточного кода на предмет его производительности лучше проверить производительность в как можно большем количестве систем с разной конфигурацией. Это позволит получить целостное представление о масштабируемости. По крайней мере следует прогнать тесты на однопроцессорном компьютере и на доступной вам машине с максимальным количеством процессоров.

Резюме

В этой главе мы рассмотрели различные типы ошибок, с которыми можно столкнуться при использовании конкурентности, — от взаимных и активных блокировок до состояний гонки за данными и другими проблематичными состояниями гонки. Были описаны приемы обнаружения ошибок, а также факторы, требующие осмысления при визуальном анализе кода. Я дал рекомендации по разработке кода с прицелом на удобство тестирования и предложил несколько приемов структуризации тестов для конкурентного кода. И в заключение мы рассмотрели некоторые полезные компоненты, способные помочь в процессе тестирования.

Приложения